

Item 1 — Adapters determinísticos respeitam o input propagado — Relatório

Data: 2026-08-10 · **Projeto:** ClinIA — Clínica Médica Inteligente · **Executor:** Claude (correção no gerador do LangNet) · **Commit:** 4616d34

Primeiro item do "fluxo clínico de ponta a ponta": os adapters determinísticos gerados pelo LangNet passam a **usar o contexto propagado** (o atendimento corrente) em vez de re-derivar os FKs por lookups frágeis. Isso destrava a etapa de **Encaminhamento** pela UI real. Feito e provado no banco.

1. O problema (raiz encontrada)

O LangNet gera, para tarefas de persistência, funções `<task>_deterministic(input_data)` a partir dos passos SQL escritos na descrição da task (`_emit_sql_step`). O adapter `criar_encaminhamento` vinha com **dois defeitos combinados**:

1. **Lia sempre** `_row['id']` na captura de um SELECT — então `SELECT atendimento_id FROM prontuarios` era guardado como `None` (a coluna nem era `id`).
2. **Derivava o FK por lookup circular/frágil** — buscava `atendimento_id` num **prontuário que ainda não existe** (o prontuário é criado depois) — e **ignorava** o `atendimento_id` que a própria UI já enviava (o **atendimento corrente** propagado da triagem).

Resultado: `criar_encaminhamento` falhava com `Column 'atendimento_id' cannot be null`.

2. A correção (geral — vale para qualquer app gerado)

No gerador (`_emit_sql_step` em `backend/agents/langnetagents.py`), a captura de um SELECT passou a:

```
# ANTES: atendimento_id = _row['id'] if _row else None
# DEPOIS: atendimento_id = input_data.get('atendimento_id') or (_row['atendimento_id'] if _row else None)
```

- **Prefere o `input_data`** (o contexto propagado, ex.: `atendimento_id` do atendimento corrente); o SELECT vira apenas **fallback**.
- **Lê a coluna realmente selecionada** (não mais `id` fixo).

Isso faz o adapter **respeitar o que a UI enviou** em vez de re-inventar a origem do FK.

3. Prova — a task REAL `criar_encaminhamento` persiste com o atendimento propagado

Reinicie o ws-server com o adapter corrigido e chamei a **task real** (não o CRUD genérico), passando o `atendimento_id` propagado da triagem + médico/especialidade:

```
atendimento_id PROPAGADO da triagem = 02b14c65-94c7-11f1-8a81-cbea323b9023
criar_encaminhamento -> { status: 'sucesso',
                           atendimento_id: '02b14c65-...' (o MESMO da triagem),
```

```
medico_id: '46640849-...',  
encaminhamento_id: '6033a1ab-...' }
```

Verificação no banco `clinia_ops` (join encaminhamento → atendimento → paciente):

```
{  
  "enc_id": "6033a1ab-94ed-11f1-8a81-cbea323b9023",  
  "atendimento_id": "02b14c65-94c7-11f1-8a81-cbea323b9023", // ← propagado da triagem  
  "medico_id": "46640849-903a-11f1-8a81-cbea323b9023",  
  "especialidade_id": "3483ffeb-903a-11f1-8a81-cbea323b9023",  
  "paciente_id": "0254cc6b-94c7-11f1-8a81-cbea323b9023",  
  "nome": "Ana Prado 780429"  
}  
→ encaminhamento.atendimento_id == atendimento propagado da triagem? TRUE
```

O FK do atendimento **viajou** da triagem até o encaminhamento persistido — agora pela **task real**, não só pelo adapter genérico.

4. Como apliquei

- Correção no **gerador** (`_emit_sql_step`) — vale para todo app gerado dali em diante.
- Reapliquei a seção determinística regenerada ao ws-server da ClinIA (`adapters.py`; backup em `adapters.py.bak`), reiniciei o ws-server e validei contra o banco.

5. Conclusão e próximo item

- **Item 1 FEITO e comprovado:** adapters determinísticos usam o **atendimento corrente propagado**; a etapa de **Encaminhamento** persiste corretamente pela task real.
- **Próximo (item 2):** a **Geração de Pré-diagnóstico** é agêntica e hoje **não persiste** o registro — falta ela **salvar o pre_diagnostico e devolver o pre_diagnostico_id** (write-back), para o **Prontuário** (que exige `pre_diagnostico_id + encaminhamento_id`) fechar a cadeia de ponta a ponta.